

Presenter & Speaker Notes

Activation Oracles pipeline — Steps 1–3, training background, and notebook code

Steps 1 & 2 — collect activations, build the oracle prompt

[steps_1_2.svg](#)

Target model & Step 1

- Target model runs a completely normal forward pass (layers 0–35) — unaware an oracle exists
- Hook fires at layer 18/36 (50% depth), capturing activations **[seq_len × 4096]** — the entire signal the oracle sees, raw and uncompressed (no PCD-style bottleneck)
- `collect_activations_multiple_layers` hooks `model.model.layers[18]`, stores `outputs[0]`, raises `EarlyStopException` to skip the rest of the pass
- 50%-depth is the paper's default (Appendix C.5): robust across layers 25–75%, poor in early layers (0–10%)

Step 2 — build the oracle prompt

- `get_introspection_prefix(layer=18, num_positions=5)` builds `"Layer: 18\n ? ? ? ? ? \n"`, prepended to the question — matches Section 3.1's input structure
- `find_pattern_in_tokens` scans for `" ?"` token IDs, verifies count & consecutiveness, returns positions — carried into `OracleInput.positions`, used by the Layer 1 hook

Connecting to the paper

- Captured activations are exactly the $\mathbf{v}$ vectors in $\mathbf{h}'_i = \mathbf{h}_i + \|\mathbf{h}_i\| \cdot (\mathbf{v}_i / \|\mathbf{v}_i\|)$ — Steps 1–2 gather the ingredients; injection happens later, at layer 1

Step 3 — package into OracleInput

steps_3_with_formula.svg

What's on the slide

- `create_oracle_input(prompt, layer, n, tokenizer, acts_BD)` → `OracleInput`: `input_ids` (tokenised prompt), `steering_vectors` (activations, CPU + detached), plus `layer`, `positions`

The handoff step

- Packages Step 1's activations and Step 2's oracle prompt into one object the oracle consumes — pure packaging, not a transformation
- Detached from the autograd graph, cloned to CPU — no gradient connection back to the target model, which is what makes injection safe

Why a dataclass, and what `positions` carries

- Gives every downstream function (steering hook, `run_oracle`) one unambiguous contract instead of four loose arguments
- `positions` = the `?` token indices from Step 2 — tells the Layer 1 hook exactly which residual-stream positions to modify

Connecting to the paper

- `steering_vectors` are the $\mathbf{v}$ vectors in Eq. 1; `OracleInput` carries $\mathbf{v}$, layer, and positions forward — a container, not a compressor (unlike PCD's encoder)

Background — target/suspect models & oracle training

Target vs. suspect models

- **"Target model"** = the general role, whoever's activations get captured and fed to the oracle
- **"Suspect model"** = a target fine-tuned to hide something (secret, misalignment), used for the auditing evals — same base architecture either way

How the oracles are trained

- Key asymmetry: trained only on clean base-model activations — never on suspect-model activations — yet still generalizes to auditing them
- LoRA adapter ($r=64$, attention + MLP projections) on the *same base architecture* as the target it will audit
- Trained for: Llama-3.3-70B-Instruct, Gemma-2-9B-IT, Qwen3-8B, Claude Haiku 3.5 (Haiku differs — full fine-tune, not LoRA)

Training data — 3 task types, mixed

- **SPQA** — infer system-prompt properties from user-assistant conversation activations
- **Classification** — 7 binary NLP tasks (sentiment, truth value, language ID, NER, gender, tense, entailment)
- **Self-supervised context prediction** — predict surrounding tokens from an activation sequence, no input/target overlap, scales to any text corpus

Scale & compute

- ~1M training examples; activations from 3 depths (25/50/75%) of the *base* model; evaluated at 50% depth
- Compute: 10 H100-hrs (Qwen3-8B) · 90 H200-hrs (Llama-3.3-70B)

Notebook code — Step 1 box

collect_activations_multiple_layers

```
def collect_activations_multiple_layers(
    model, submodules, inputs_BL, start_offset, end_offset,
):
    """
    Collect activations from multiple layers using forward hooks.

    Returns: Dict mapping layer → activations tensor [batch, length, d_model]
    """
    ...

    def gather_target_act_hook(module, inputs, outputs):
        layer = module_to_layer[module]

        # Handle different output formats
        if isinstance(outputs, tuple):
            activations_BLD_by_layer[layer] = outputs[0]
        else:
            activations_BLD_by_layer[layer] = outputs

        # Early stop after max layer
        if layer == max_layer:
            raise EarlyStopException("Early stopping after capturing activations")

    # Register hooks
    handles.append(submodule.register_forward_hook(gather_target_act_hook))
```

Notebook code — Step 2 box

get_introspection_prefix

```
def get_introspection_prefix(layer: int, num_positions: int) -> str:
    """Create the prefix for oracle prompts with ? tokens."""
    prefix = f"Layer: {layer}\n"
    prefix += SPECIAL_TOKEN * num_positions
    prefix += " \n"
    return prefix
```

find_pattern_in_tokens

```
def find_pattern_in_tokens(token_ids, special_token_str, num_positions, tokenizer):
    """
    Find positions of special token in tokenized sequence.
    Returns: List of positions where special token appears
    """
    ...
    # Find ALL positions where this token appears (don't stop early)
    positions = [i for i, tid in enumerate(token_ids) if tid == special_token_id]

    if len(positions) != num_positions:
        raise ValueError(f"Expected {num_positions} occurrences...")

    assert positions[-1] - positions[0] == num_positions - 1, "Positions are not consecutive"

    return positions
```

Notebook code — Step 3 box

create_oracle_input

```
def create_oracle_input(prompt, layer, num_positions, tokenizer, acts_BD) -> OracleInput:
    """
    Create an oracle input for inference.
    Returns: OracleInput ready for generation
    """
    ...
    input_prompt_ids = tokenizer.apply_chat_template(
        input_messages, tokenize=True, add_generation_prompt=True,
        return_tensors=None, padding=False, enable_thinking=False,
    )

    # Find ? token positions in the prompt
    positions = find_pattern_in_tokens(input_prompt_ids, SPECIAL_TOKEN, num_positions, tokenizer)

    # Ensure activations are on CPU and detached
    acts_BD = acts_BD.cpu().clone().detach()

    return OracleInput(
        input_ids=input_prompt_ids, layer=layer,
        steering_vectors=acts_BD, positions=positions,
    )
```

Notebook code — Layer 1 steering hook

get_hf_activation_steering_hook

```
def get_hf_activation_steering_hook(vectors, positions, steering_coefficient, device, dtype):  
    """  
    Create hook that injects activations at specified positions (assumes batch_size=1).  
    Returns: Hook function that modifies activations during forward pass  
    """  
    # Normalize vectors to unit norm  
    normed_vectors = torch.nn.functional.normalize(vectors, dim=-1).detach()  
    ...  
    def hook_fn(module, _input, output):  
        ...  
        # Get original activations at steering positions  
        orig_KD = resid_BLD[0, positions_tensor, :]          # [K, d_model]  
        norms_K1 = orig_KD.norm(dim=-1, keepdim=True)        # [K, 1]  
  
        # Scale normalized steering vectors by original magnitudes  
        steered_KD = (normed_vectors * norms_K1 * steering_coefficient).to(dtype)  
  
        # Inject (add to original)  
        resid_BLD[0, positions_tensor, :] = steered_KD.detach() + orig_KD
```

Mapping to the diagram's pseudocode: `orig = orig_KD` , `norms = norms_K1` , `normed_v = normed_vectors` , `steered = steered_KD` . `normed_vectors` is `v_i/||v_i||` ; `norms_K1` is `||h_i||` ; the last line is the addition `h_i + steered` .